

情報通信工学実験I

テーマ: アセンブリ言語

MIPS ISAの浮動小数点演算

1～2回目のMIPSアセンブリ言語演習

- 基本算術演算・論理演算・分岐命令
- メモリアクセス・ループ・手続き

整数(符号を意識
しなかった)

本日の演習内容: 符号付き整数と小数の処理

- 2の補数で符号付き整数の処理
- IEEE754で浮動小数点数の処理

符号付き整数の表現

32ビット整数の表現範囲

- 符号なし : $0 \sim 2^{32}-1$
- 符号付き(2の補数) : $-2^{31} \sim 2^{31}-1$

32ビット	
符号	31ビット

数値	符号付絶対値 (符号Bit+絶対値)	1の補数 (全ビット反転)	2の補数 (全ビット反転+1)
3	0 1 1	0 1 1	0 1 1
2	0 1 0	0 1 0	0 1 0
1	0 0 1	0 0 1	0 0 1
0	0 0 0	0 0 0	0 0 0
-0	1 0 0	1 1 1	-
-1	1 0 1	1 1 0	1 1 1
-2	1 1 0	1 0 1	1 1 0
-3	1 1 1	1 0 0	1 0 1
-4	-	-	1 0 0

2の補数表現のメリット

- 負の0の表現がなく、表現空間をもったいなく利用できる
- 足し算で引き算を実現できる→加算器だけを実装すればよい

2の補数を用いた計算例

$$7 - 9 = ?$$

$$7_{10} = \textcolor{red}{0}000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0111_2$$

$$\begin{aligned} -9_{10} &= \sim(\textcolor{red}{0}000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 1001_2) + 1 \\ &= \textcolor{red}{1}111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 0110_2 + 1 \\ &= \textcolor{red}{1}111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 0111_2 \end{aligned}$$

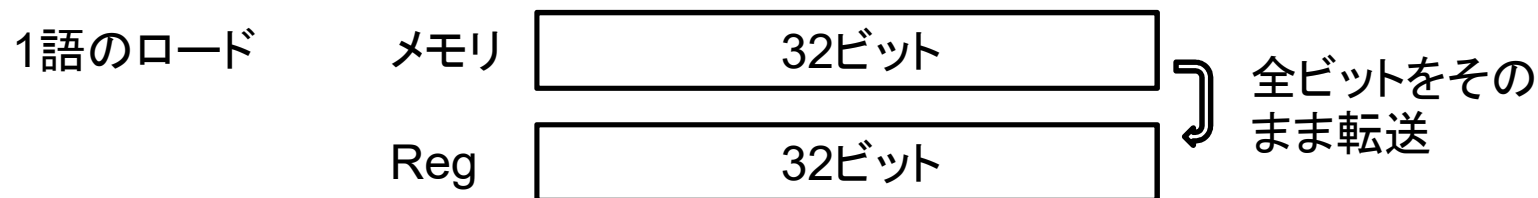
$$7 - 9 = 7 + (-9) =$$

$$\begin{array}{r} \textcolor{red}{0}000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0111_2 \\ + \textcolor{red}{1}111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 0111_2 \\ \hline \textcolor{red}{1}111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1110_2 = -2 \end{array}$$

- 符号付き・符号なし(**u**nsigned)命令を別々に用意

- 算術演算: add/addu, addi/addiu, sub/subu
- 比較: slt/sltu, slti/sltiu
- ロード: lb/lbu (load byte unsigned), lh/lhu (load halfword unsigned)
1語より短いデータをロード時に上位ビットを拡張するため

符号を意識しながら、
正しい命令を使うべき



符号によって、残りの24ビットを0か1で埋める(**符号拡張**)

例 $2_{10} = 0000\ 0010_2 \rightarrow 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0010_2$
 $-2_{10} = 1111\ 1110_2 \rightarrow 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1110_2$

課題3-1: 符号付き命令の演習

- 下記のプログラム(Moodle の ex3-1.asm)を実行する。
- レジスタの値を観測し、問1～問3に理由を付して答えなさい。

```
ex3-1.asm
1  data
2  const0: .byte -2
3
4  .text
5  li      $s0, 7
6  li      $s1, 9
7  li      $s2, -9
8
9  sub     $t0, $s0, $s1
10 add     $t1, $s0, $s2
11
12 slt     $t2, $s0, $s2
13 sltu    $t3, $s0, $s2
14
15 la      $t8, const0
16 lb      $t4, 0($t8)
17 lbu     $t5, 0($t8)
```

問1: \$t0と\$t1は同じか?

問2: \$t2と\$t3は同じか?

問3: \$t4と\$t5は同じか?

問1: t_0 と t_1 は同じか？理由を付して述べよ。

問2: t_2 と t_3 は同じか？理由を付して述べよ。

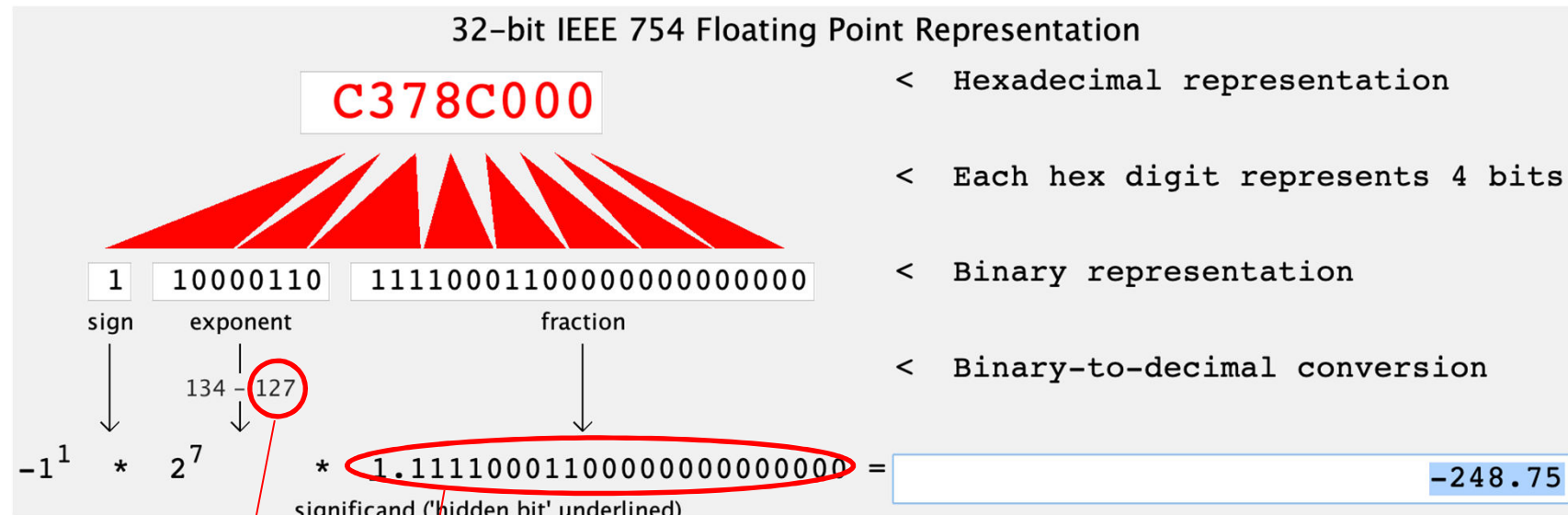
問3: t_4 と t_5 は同じか？理由を付して述べよ。

浮動小数点数の規格: IEEE754

符号 s	指数部 e	仮数部 f
1b	8b	23b

$$N = (-1)^s * (1+f) * 2^{e-\text{bias}}$$

(例1) IEEE754表現から10進数へ変換:



Bias: 8ビットの2の補数eの範囲は-126~127、数字の比較を簡単にするため、+127のBiasで範囲を1~254に調整する(倍精度ならBias=1023)

1(必ずあるから仮数部で省略) + $2^{-1} + 2^{-2} + 2^{-3} + 2^{-4} + 2^{-8} + 2^{-9} = 1.94335938$

浮動小数点数の規格: IEEE754

符号 s	指数部 e	仮数部 f
1b	8b	23b

$$N = (-1)^s * (1+f) * 2^{e-\text{bias}}$$

(例1) 10進数-248.75のIEEE754表現を求める

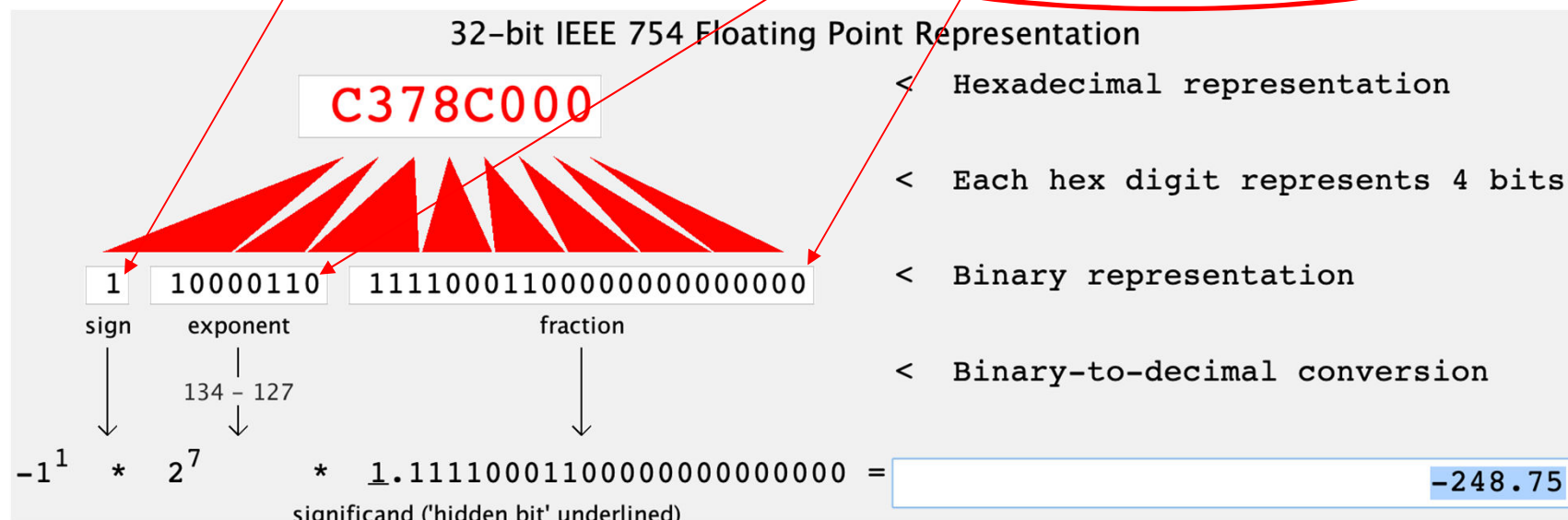
符号s: 負数だから、s = 1

指数部e: $\log_2 248.75 \doteq 7.9$ 、整数部だけ取ると power=7

power=7の場合、 $e = 7 + 127 = (134)_{10} = (1000110)_{2}$

仮数部f: $f = 248.75 / 2^7 - 1 \doteq (0.94335938)_{10} = 0.111100011000000000000000_{2}$

有限桁の浮動小数点数は離散的に存在するため、10進→2進変換に伴う丸め誤差や、計算時の丸め誤差、打ち切り誤算が発生します。



- ALUではなく、専用のFPU(Floating-point unit)処理
 - 32本の32ビット浮動小数点レジスタがある(\$f0~\$f31)
 - 倍精度(64ビット)は2本の32ビットで表現
名前は、偶数番目
- 単精度の処理命令(一部)
 - 算術演算: add.s, sub.s, div.s, mul.s
 - ロード/ストア: l.s (疑似命令), lwc1, swc1
 - レジスタ間転送: mov.s
 - 整数との変換: cvt.w.s (整数←単精度),
cvt.s.w (単精度←整数)

詳細は、「32MIPSの浮動小数点処理命令.pdf」を参照。

Marsの浮動小数点数ツール

Coproc 1を選択したらFPLレジスタを確認できる

IEEE754ツールで簡単にFPLレジスタの10進数値を確認できる

The screenshot shows the Mars MIPS simulator interface. The 'Tools' menu is open, and 'Floating Point Representation' is selected. The 'Floating Point Representation, Version 1.1' window is displayed, showing the 32-bit IEEE 754 Floating Point Representation of the value 42000000. The window includes a diagram of the IEEE 754 format with sign, exponent, and fraction fields. Below the diagram, the binary representation is shown as 0 1000100 000000000000000000000000. The binary-to-decimal conversion is shown as $-1^0 \times 2^5 \times 1.000000000000000000000000 = 32.0$. The MIPS floating point Register of interest is set to \$f4. The Coproc 1 register window is also visible, showing the value 42000000 in the \$f4 register.

32-bit IEEE 754 Floating Point Representation

42000000

< Hexadecimal representation

< Each hex digit represents 4 bits

< Binary representation

< Binary-to-decimal conversion

sign exponent fraction

0 1000100 000000000000000000000000

132 - 127

$-1^0 \times 2^5 \times 1.000000000000000000000000 = 32.0$

significand ('hidden bit' underlined)

Instructions

The program and register \$f4 will respond to each other when MIPS program connected or running.

MIPS floating point Register of interest: \$f4

Tool Control

Disconnect from MIPS Reset Close

Registers Coproc 1 Coproc 0

Name	Float	Double
\$f0	0x00000000	0x0000000000000000
\$f1	0x00000000	0x0000000000000000
\$f2	0x00000000	0x0000000000000000
\$f3	0x00000000	0x0000000000000000
\$f4	0x42000000	0x0000000042000000
\$f5	0x00000000	0x0000000000000000
\$f6	0x00000000	0x0000000000000000
\$f7	0x00000000	0x0000000000000000
\$f8	0x00000000	0x0000000000000000
\$f9	0x00000000	0x0000000000000000
\$f10	0x00000000	0x0000000000000000
\$f11	0x00000000	0x0000000000000000
\$f12	0x41200000	0x0000000041200000
\$f13	0x00000000	0x0000000000000000
\$f14	0x00000000	0x0000000000000000
\$f15	0x00000000	0x0000000000000000
\$f16	0x00000000	0x0000000000000000
\$f17	0x00000000	0x0000000000000000
\$f18	0x00000000	0x0000000000000000
\$f19	0x00000000	0x0000000000000000
\$f20	0x00000000	0x0000000000000000
\$f21	0x00000000	0x0000000000000000
\$f22	0x00000000	0x0000000000000000
\$f23	0x00000000	0x0000000000000000
\$f24	0x00000000	0x0000000000000000
\$f25	0x00000000	0x0000000000000000
\$f26	0x00000000	0x0000000000000000
\$f27	0x00000000	0x0000000000000000
\$f28	0x00000000	0x0000000000000000
\$f29	0x00000000	0x0000000000000000
\$f30	0x00000000	0x0000000000000000
\$f31	0x00000000	0x0000000000000000

課題3-2: 華氏温度→摂氏温度の変換プログラム

- 華氏温度から摂氏温度への変換プログラム(ex3-2.asm)を実行し、以下の3つの問に解答しなさい。

```
ex3-2.asm
1  .data
2  input:  .float 10.0
3  const0: .word 32
4  const1: .float 0.555555
5
6  .text
7  l.s     $f2, input
8
9  # celsius = (fahrenheit - 32.0) * 5.0 / 9.0
10 l.s     $f4, const0
11 cvt.s.w $f4, $f4
12 l.s     $f6, const1
13 sub.s   $f0, $f2, $f4
14 mul.s   $f0, $f0, $f6
```

問1: このプログラムが何を行っているか理解し、コメントを付けなさい。

5.0/9.0を定数として設定すると、計算の命令が減らせる

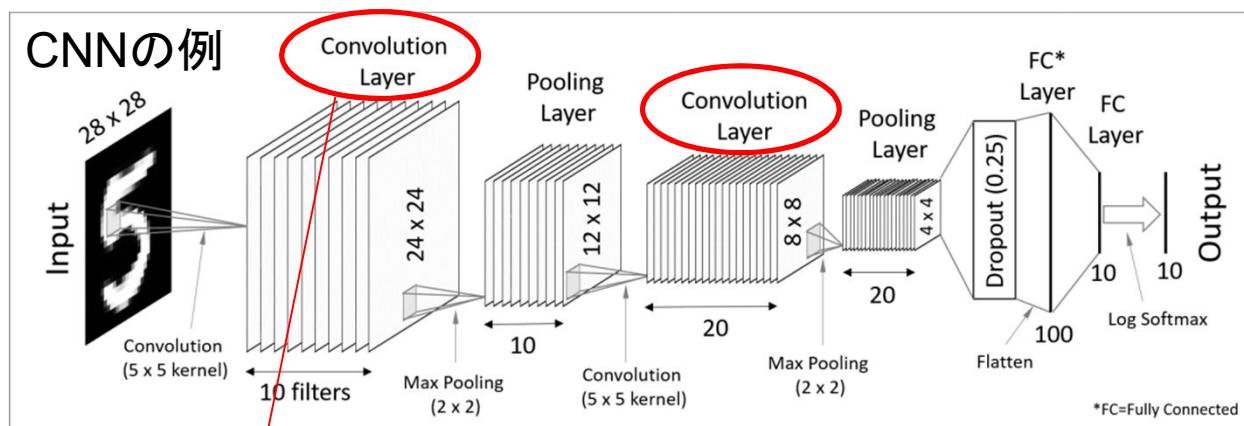
問2: \$f0にあるIEEE754浮動小数点数の結果を手動で10進数に変換し、変換の手順も示しなさい。

問3: \$f0にある結果と手計算の結果を比較しなさい。誤差があるならその原因を説明しなさい。

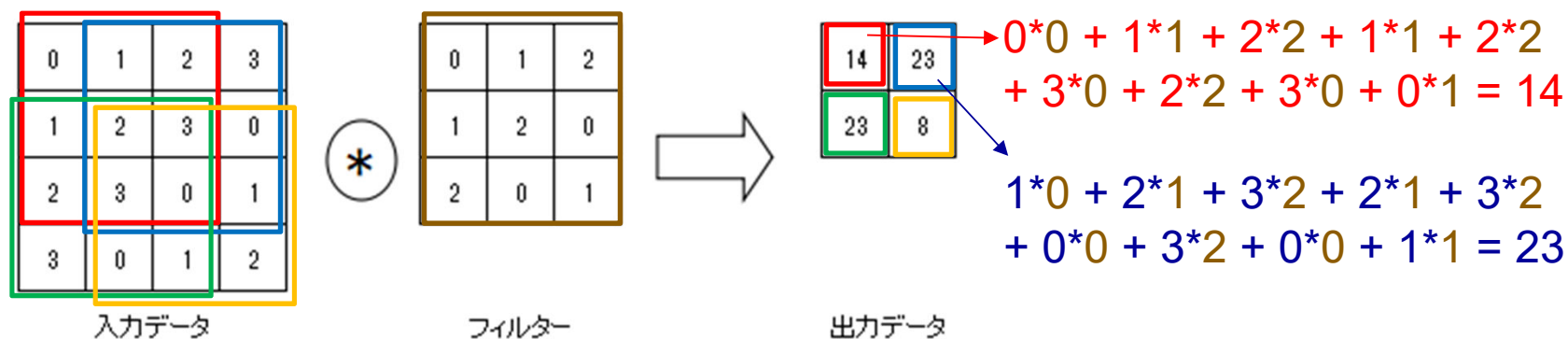
- 問1: このプログラムが何を行っているか理解し、コメントを付けなさい。
- 問2: 実行終了後の\$`f0`にあるIEEE754浮動小数点数の結果を手動で10進数に変換し、変換の手順も示しなさい。
- 問3: 上記の\$`f0`にある結果と手計算の結果を比較しなさい。誤差があるならその原因を説明しなさい。

課題3-3: 積和計算プログラム

- CNN(畳み込みニューラルネットワーク)は画像認識などの分野において、よく使われているAIネットワーク
- その計算のコア関数は入力行列とフィルタ行列の積和演算



CNNでの積和演算の例



課題3-3: 積和計算プログラム

課題: ex3-3をベースにして、下記の積和演算を行うプログラムを作成する。

0.0	1.2	2.5
1.1	2.1	3.1
2.1	3.1	0.0

⊗

0.0	1.0	2.0
1.0	2.0	0.0
2.0	0.0	1.0

条件: 下記のデータセグメントを利用し、結果をoutputに保存する
(ex3-3.asmに記入済み)

```
1  .data
2  size: .word 9
3  input:  .float 0.0, 1.2, 2.5, 1.1, 2.1, 3.1, 2.1, 3.1, 0.0
4  filter: .float 0.0, 1.0, 2.0, 1.0, 2.0, 0.0, 2.0, 0.0, 1.0
5  output: .float 0.0
```


課題3-3: 積和計算プログラム

ヒント:

- メインのループ (size 回以下を繰り返す)

$\$f3 \leftarrow \text{input} * \text{filter}$

$\$f3$ を $\$f4$ に足し込んで行く

次のループへの準備

- 最初のデータのロード

la \$t0, input

lwc1 \$f1, 0(\$t0)

- 次のデータのロード

addi \$t0, \$t0, 4

lwc1 \$f1, 0(\$t0)

- 最終結果のストア

la \$t4, output

swc1 \$f4, 0(\$t4)

- 結果の出力

mov.s \$f12, \$f4

li \$v0, 2

syscall

引数レジスタは \$f12

浮動小数点数の出力

- 問1: ex3-3をベースにして作成したプログラム全体を記載しなさい。
- 問2: 実行結果の値を記載しなさい(10進数で)。